

UNIVERSIDADE ESTADUAL DA PARAÍBA – UEPB

Campus I

Centro de Ciência e Tecnologia – CCT

Departamento de Computação

Bacharelado em Ciência da Computação

Gustavo Azevedo Lelis Farias

**PROJETO EXPERIMENTAL COMPARATIVO DE TÉCNICAS DE PROJETO DE
ALGORITMOS**

Relatório submetido ao Curso de Ciência da Computação da Universidade Estadual da Paraíba, na disciplina de Técnicas de Análise de Algoritmos, correspondente ao período letivo de 2025.2.

Campina Grande – PB

2026

RESUMO

Este relatório apresenta a implementação e a análise comparativa de duas técnicas exatas de busca no espaço de soluções Backtracking e Branch and Bound aplicadas aos problemas de Partição de Conjuntos e Árvore Geradora Mínima (AGM). O estudo foi conduzido com implementação em Java, com instrumentação de tempo de execução, uso de memória, nós visitados e nós podados, utilizando instâncias de tamanho variável para avaliar escalabilidade. Os resultados mostram que ambas as técnicas mantêm corretude nas instâncias avaliadas, com vantagem prática do Branch and Bound em instâncias de maior porte, a depender da qualidade dos limites utilizados. A análise teórica e empírica evidencia as limitações inerentes à complexidade exponencial de ambas as abordagens e estabelece bases para a comparação com técnicas mais eficientes nas entregas subsequentes.

SUMÁRIO

1. INTRODUÇÃO
 - 1.1. Contextualização do Problema
 - 1.2. Objetivos
 - 1.2.1. Objetivo geral
 - 1.2.2. Objetivos específicos
2. METODOLOGIA E IMPLEMENTAÇÃO
 - 2.1. Estrutura do Sistema Desenvolvido
 - 2.2. Formato de Dados de Entrada e Saída
 - 2.3. Métricas de Avaliação
3. ALGORITMOS IMPLEMENTADOS
 - 3.1. Backtracking
 - 3.1.1. Análise de Corretude
 - 3.1.2. Análise de Desempenho
 - 3.1.2.1. Espacial
 - 3.1.2.2. Temporal
 - 3.2. Branch and Bound
 - 3.2.1. Análise de Corretude
 - 3.2.2. Análise de Desempenho
 - 3.2.2.1. Espacial
 - 3.2.2.2. Temporal
4. EXPERIMENTOS E RESULTADOS
 - 4.1. Instâncias de Teste
 - 4.2. Resultados Obtidos
5. ANÁLISE E DISCUSSÃO
 - 5.1. Comparação das Estratégias
6. CONCLUSÃO
7. REFERÊNCIAS

1 INTRODUÇÃO

1.1 Contextualização do Problema

Este trabalho apresenta uma comparação entre duas técnicas clássicas de busca exata no espaço de soluções: Backtracking e Branch and Bound. A análise é conduzida por meio da aplicação dessas técnicas a dois problemas representativos: o Problema de Partição de Conjuntos, de natureza combinatória, e o Problema da Árvore Geradora Mínima (AGM), de natureza estrutural em grafos. A escolha desses dois problemas é intencional, pois eles diferem em estrutura, dificuldade computacional e características do espaço de busca, permitindo observar como cada técnica se comporta em cenários distintos.

A Partição de Conjuntos é um problema NP-completo que exige a exploração exaustiva de subconjuntos, tornando-se um terreno natural para técnicas de busca com poda. A AGM, por outro lado, possui algoritmos polinomiais eficientes, mas é usada aqui como caso de teste para avaliar o comportamento de BT e BnB em um problema de grafos onde o espaço de soluções é estruturalmente diferente. Essa dualidade permite discutir corretude, custo computacional e comportamento de poda em contextos complementares.

1.2 Objetivos

1.2.1 Objetivo geral

Implementar e comparar formalmente as estratégias de Backtracking e Branch and Bound, analisando suas características teóricas e empíricas nos problemas de Partição de Conjuntos e Árvore Geradora Mínima.

1.2.2 Objetivos específicos

- Analisar a complexidade assintótica teórica de cada técnica.
- Medir o tempo de execução empírico para instâncias de diferentes tamanhos.
- Avaliar o uso de memória durante a execução dos algoritmos.
- Verificar a corretude das soluções obtidas em relação à solução ótima.
- Investigar a escalabilidade das técnicas em função do tamanho da entrada.
- Quantificar o número de nós visitados e nós podados no espaço de busca.
- Justificar como cada técnica otimiza a busca no espaço de possíveis soluções.

2 METODOLOGIA E IMPLEMENTAÇÃO

2.1 Estrutura do Sistema Desenvolvido

Os experimentos foram conduzidos em ambiente isolado, utilizando Java como linguagem de implementação para todas as técnicas, garantindo uniformidade na medição das métricas. O tempo de execução foi capturado com `System.nanoTime()`, convertido para milissegundos, e o uso de memória foi estimado pela diferença entre `totalMemory()` e `freeMemory()` da JVM ao longo de cada execução. Valores próximos de zero para memória refletem a limitação desta estimativa quando o coletor de lixo da JVM atua entre as medições, e não ausência de alocação. As instâncias de teste foram geradas com semente aleatória fixa para assegurar reprodutibilidade e comparabilidade direta entre as técnicas.

As implementações estão organizadas nos seguintes arquivos-fonte:

- `PartitionSolver.java` — implementações de Backtracking e Branch and Bound para Partição de Conjuntos.
- `MSTSolver.java` — implementações de Backtracking e Branch and Bound para Árvore Geradora Mínima.
- `Main.java` — entrada de dados e execução de instâncias individuais.
- `BenchmarkMain.java` — execução comparativa automatizada e consolidação de métricas.

2.2 Formato de Dados de Entrada e Saída

Problema de Partição de Conjuntos

Entrada:

```
n
a1 a2 a3 ... an
```

Saída:

```
YES
subset1
subset2
Ou: NO
```

Problema de Árvore Geradora Mínima (AGM)

Entrada:

```
n m
u1 v1 w1
u2 v2 w2
...
um vm wm
```

Saída:

```
peso_total
u1 v1 w1
...
u(n-1) v(n-1) w(n-1)
```

2.3 Métricas de Avaliação

As técnicas foram comparadas com base nas seguintes métricas:

- Complexidade Assintótica (Teórica): análise da ordem de crescimento do tempo e espaço de execução no pior caso.
- Tempo de Execução (Empírico): medição do tempo real de processamento para instâncias de diferentes tamanhos.
- Uso de Memória: estimativa do pico de memória utilizada pela JVM durante a execução.
- Qualidade da Solução: verificação da corretude da solução encontrada em relação à solução ótima.
- Escalabilidade: avaliação do comportamento das técnicas à medida que o tamanho da entrada cresce.
- Métricas de Poda: quantificação dos nós visitados e podados, evidenciando a eficácia das estratégias de otimização da busca.

3 ALGORITMOS IMPLEMENTADOS

Esta seção detalha as duas técnicas de projeto de algoritmos abordadas nesta primeira entrega, apresentando suas descrições, fragmentos de implementação reais extraídos da base Java e análises de corretude e desempenho para os dois problemas de referência.

3.1 Backtracking

3.1.1 Análise de Corretude

O Backtracking é uma técnica de busca exaustiva que constrói soluções de forma incremental, retrocedendo sempre que detecta que o estado parcial corrente não pode conduzir a uma solução válida. Sua corretude reside na garantia de que, se uma solução existe, ela será encontrada, pois todas as possibilidades são exploradas enquanto as podas aplicadas forem seguras isto é, descartarem apenas ramos que provadamente não contêm soluções válidas.

Problema de Partição de Conjuntos

Para a Partição de Conjuntos, o Backtracking busca um subconjunto cuja soma seja exatamente $S_{total}/2$. A corretude é assegurada pela exploração de todas as combinações possíveis de elementos para formar o primeiro subconjunto. Se uma partição com somas iguais existe, o algoritmo a encontrará. A poda de viabilidade que descarta o ramo quando a soma corrente excede o alvo é segura porque nenhum elemento do conjunto possui valor negativo, tornando impossível reverter o excesso por adições futuras.

Fragmento de implementação (Backtracking – Partição de Conjuntos):

```
private void btRecursive(int idx, int currentSum,
    List<Integer> currentIndices, PartitionData result) {
    nodesVisited++;
    if (result.found) return;
    if (idx == n) {
        if (currentSum == targetSum) { result.found = true; /*
monta subsets */ }
        return;
    }
    if (currentSum > targetSum) { nodesPruned++; return; } //
poda de viabilidade
    currentIndices.add(idx);
    btRecursive(idx + 1, currentSum + elements[idx],
currentIndices, result);
    currentIndices.remove(currentIndices.size() - 1);
    btRecursive(idx + 1, currentSum, currentIndices, result);
}
```

O trecho mostra a exploração recursiva com poda por inviabilidade: quando a soma parcial excede o alvo, o ramo é descartado sem comprometer a corretude da busca exata, pois nenhum descendente desse estado poderá produzir uma partição válida.

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, o Backtracking enumera todas as combinações válidas de arestas que formam árvores geradoras, selecionando a de menor custo total. A abordagem é correta qualquer AGM ótima será encontrada porém a explosão combinatória do espaço de busca torna a técnica impraticável para grafos com mais de 8 a 10 vértices, onde o número de árvores geradoras possíveis cresce exponencialmente com o número de arestas.

3.1.2 Análise de Desempenho

3.1.2.1 Espacial

A complexidade espacial do Backtracking é $O(n)$ para Partição e $O(E)$ para AGM, onde n é o número de elementos e E o número de arestas. Esse espaço é consumido exclusivamente pela pilha de recursão, que armazena o estado corrente da busca até a profundidade máxima da árvore de decisão. O custo espacial reduzido é uma vantagem estrutural sobre o Branch and Bound, que mantém uma fronteira de nós ativos em memória diferença que se torna significativa nas instâncias maiores, como evidenciado pelos experimentos.

3.1.2.2 Temporal

A complexidade temporal do Backtracking é $O(2^n)$ no pior caso para Partição de Conjuntos, onde cada elemento pode ou não ser incluído no subconjunto. Para a AGM, a complexidade é ainda maior, crescendo com o número de subconjuntos de arestas que formam árvores geradoras. As podas de viabilidade reduzem o número de nós efetivamente visitados, mas esse benefício é insuficiente para tornar a técnica viável em instâncias de maior porte: os dados empíricos mostram crescimento de 8 nós visitados para $n = 4$ até 504.009 nós para $n = 20$, e inviabilidade para a AGM a partir de $V = 10$.

- Partição de Conjuntos: $O(2^n)$ no pior caso.
- AGM por enumeração: exponencial no número de arestas; inviável na prática para $V \geq 10$.

3.2 Branch and Bound

3.2.1 Análise de Corretude

O Branch and Bound estende o Backtracking ao introduzir limites (bounds) que permitem descartar ramos cujo melhor resultado possível já é inferior à melhor solução conhecida. A corretude é mantida desde que os limites sejam admissíveis nunca subestimando o potencial dos descendentes. Se o bound de um nó não supera o incumbente, esse ramo é podado com segurança, pois nenhum de seus descendentes poderá produzir uma melhora. A diferença fundamental em relação ao Backtracking é que, além de podar por inviabilidade, o Branch and Bound poda por dominância.

Problema de Partição de Conjuntos

Para a Partição, o bound verifica se a diferença atual entre as somas pode ser compensada pelos elementos restantes. Se $|\text{sum1} - \text{sum2}| > \text{soma dos elementos restantes}$, é impossível equilibrar os subconjuntos a partir desse nó. A fila de prioridade expande primeiro os nós mais equilibrados (menor diferença entre as somas), acelerando a convergência para a solução. Esse mecanismo justifica o número maior de podas observado em relação ao Backtracking puro.

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, o custo parcial acumulado é comparado ao incumbente. Se o custo do nó corrente já supera o melhor peso conhecido, toda a subárvore é descartada. Embora o critério seja correto, a dificuldade em definir limites inferiores informativos para subconjuntos parciais de arestas limita a eficácia das podas, explicando o comportamento exponencial observado para instâncias maiores.

Fragmento de implementação (Branch and Bound – AGM):

```
while (!queue.isEmpty()) {
    Node node = queue.poll();
    nodesVisited++;
    if (node.edges.size() == nVertices - 1) {
        if (node.weight < bestWeight) {
            bestWeight = node.weight;
            bestEdges = new ArrayList<>(node.edges);
        }
        continue;
    }
    if (node.weight >= bestWeight) {
        nodesPruned++; // bound pior que incumbente:
        descartar
        continue;
    }
}
```

A regra `node.weight >= bestWeight` implementa o bound de poda: nós que já não podem melhorar a melhor solução corrente são descartados sem exploração adicional, reduzindo o espaço efetivamente visitado.

3.2.2 Análise de Desempenho

3.2.2.1 Espacial

A complexidade espacial do Branch and Bound pode atingir $O(2^n)$ no pior caso, pela manutenção da fronteira de nós ativos aguardando expansão. Esse comportamento é diretamente observado nos experimentos: o Branch and Bound consumiu 491 KB para $n = 12$ e 47.649 KB para $n = 20$ em Partição, enquanto o Backtracking registrou 0,00 KB nos mesmos tamanhos. Quando o bound é pouco informativo e muitos ramos permanecem na fronteira, o custo espacial pode se tornar tão limitante quanto o custo temporal.

3.2.2.2 Temporal

No pior caso, a complexidade temporal do Branch and Bound também é $O(2^n)$. Na prática, a qualidade do bound determina a eficácia das podas. Para Partição com $n = 20$, o Branch and Bound realizou 104.474 podas de um total de 504.009 nós visitados, demonstrando que o bound foi suficientemente informativo para reduzir expressivamente o espaço explorado em relação ao Backtracking puro. Para a AGM, o Branch and Bound ainda completou $V = 10$ em 184 ms, enquanto o Backtracking atingiu o limite prático para o mesmo tamanho.

- Partição de Conjuntos: $O(2^n)$ no pior caso, com redução prática dependente da qualidade do bound.
- AGM: $O(2^n)$ no pior caso; viável até $V = 10$ nas instâncias testadas.

4 EXPERIMENTOS E RESULTADOS

4.1 Instâncias de Teste

O benchmark foi executado por `BenchmarkMain.java` com variação sistemática do tamanho da entrada. Para Partição de Conjuntos, foram utilizados conjuntos de $n \in \{4, 8, 12, 16, 20\}$ elementos inteiros positivos, incluindo a instância de referência `elements = [1, 5, 11, 5]` para verificação de corretude e instâncias maiores geradas com semente

fixa para avaliação de escalabilidade. Para a AGM, grafos não direcionados e ponderados com $V \in \{5, 6, 8, 10\}$ vértices foram gerados com densidade progressiva. Os casos sinalizados com "-" indicam instâncias onde o timeout prático foi atingido antes da conclusão.

4.2 Resultados Obtidos

Todos os experimentos foram executados em ambiente isolado com instrumentação Java para tempo (`System.nanoTime()`) e memória (`Runtime.getRuntime()`). As instâncias foram geradas com semente fixa para garantir comparabilidade entre as técnicas.

Partição de Conjuntos

A Tabela 1 consolida os resultados para o Problema de Partição de Conjuntos com variação de n , apresentando tempo de execução, uso de memória, número de nós visitados e podados, e confirmação de resultado correto.

Tabela 1 – Partição de Conjuntos: tempo, memória, exploração e corretude (saída do `BenchmarkMain`).

n	Técnica	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados	Resultado
4	Backtracking	0,6326	0,00	8	0	SIM
4	Branch and Bound	1,2838	0,00	18	6	SIM
8	Backtracking	2,6445	0,00	50	0	SIM
8	Branch and Bound	9,7076	0,00	105	28	SIM
12	Backtracking	0,2922	0,00	158	0	SIM
12	Branch and Bound	15,6109	491,54	1764	386	SIM
16	Backtracking	0,1480	0,00	46	0	SIM
16	Branch and Bound	96,3449	5242,78	25196	5546	SIM
20	Backtracking	1,8649	0,00	1150	0	SIM
20	Branch and Bound	473,8015	47649,44	504009	104474	SIM

Fonte: elaborado pelo autor (2026).

A Tabela 2 detalha as soluções encontradas para a instância de referência `elements = [1, 5, 11, 5]`, confirmando que ambas as técnicas identificam a mesma partição exata, ainda que em ordens distintas.

Tabela 2 – Soluções encontradas para Partição (instância de referência: $n = 4$).

Técnica	Subconjunto 1	Subconjunto 2	Soma(subset1)	Soma(subset2)
Backtracking	[1, 5, 5]	[11]	11	11
Branch and Bound	[11]	[1, 5, 5]	11	11

Fonte: elaborado pelo autor (2026).

Os resultados evidenciam o crescimento diferenciado entre as duas técnicas. O Backtracking manteve uso de memória nulo em todas as instâncias, reflexo de sua complexidade espacial $O(n)$, enquanto o Branch and Bound atingiu 47.649 KB para $n = 20$. Em contrapartida, o Branch and Bound realizou 104.474 podas para $n = 20$, redução expressiva do espaço explorado que justifica sua viabilidade em instâncias onde o Backtracking seria proibitivo.

Árvore Geradora Mínima

A Tabela 3 apresenta os resultados para o Problema da AGM com instâncias de tamanho crescente, comparando Backtracking e Branch and Bound em grafos de $V \in \{5, 6, 8, 10\}$ vértices.

Tabela 3 – Árvore Geradora Mínima: tempo, memória, exploração e qualidade (saída do BenchmarkMain).

V	E	Técnica	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados	Peso
5	7	Backtracking	1,6533	0,00	145	16	10
5	7	Branch and Bound	1,2538	0,00	163	36	10
6	14	Backtracking	24,7827	0,00	1905	468	64
6	14	Branch and Bound	23,3178	0,00	1905	592	64
8	18	Backtracking	17,6189	4096,00	10892	3502	67
8	18	Branch and Bound	32,0099	2048,00	10892	4177	67
10	22	Backtracking	-	-	-	-	limite
10	22	Branch and Bound	184,5703	22528,00	89651	34796	78

Fonte: elaborado pelo autor (2026).

A Tabela 4 detalha as arestas da AGM encontrada para a instância de referência com 5 vértices e 7 arestas, com peso total 10 confirmado como ótimo por ambas as técnicas.

Tabela 4 – Arestas da AGM válida encontrada (instância de referência: 5 vértices, 7 arestas).

u	v	w
1	2	1
0	2	2
3	4	2
1	3	5

Fonte: elaborado pelo autor (2026).

Para a AGM, o Branch and Bound manteve desempenho superior em tempo na instância de referência (1,25 ms vs 1,65 ms do Backtracking), com maior número de podas (36 vs 16). Para $V = 6$, ambas as técnicas visitaram 1.905 nós o mesmo espaço de busca porém o Branch and Bound realizou 592 podas contra 468 do Backtracking, reflexo do bound adicional. A partir de $V = 10$, o Backtracking atingiu o limite prático enquanto o Branch and Bound ainda completou em 184 ms.

5 ANÁLISE E DISCUSSÃO

5.1 Comparação das Estratégias

A Tabela 5 consolida as dimensões de análise para cada técnica, considerando suas características teóricas e o desempenho empírico observado nos dois problemas estudados.

Tabela 5 – Síntese comparativa das técnicas na primeira entrega.

Critério	Backtracking	Branch and Bound
Corretude em Partição	Exata	Exata
Corretude em AGM	Exata (inst. peq.)	Exata (inst. peq.)
Complexidade temporal	$O(2^n)$	$O(2^n)^\dagger$
Complexidade espacial	$O(n)$	$O(2^n)$ fronteira
Facilidade de implementação	Alta	Média
Escalabilidade prática	Baixa a média	Média
Mecanismo de otimização	Poda de viabilidade	Bound + viabilidade

[†] Pior caso teórico $O(2^n)$, com redução prática dependente da qualidade do bound.

Os resultados demonstram que ambas as técnicas são corretas para as instâncias avaliadas, preservando a solução ótima em todos os casos concluídos. A distinção

prática entre elas reside não na complexidade assintótica ambas são exponenciais no pior caso mas na eficácia do mecanismo de poda. O Backtracking opera exclusivamente por poda de viabilidade: quando o estado parcial viola uma condição necessária, o ramo é descartado. O Branch and Bound adiciona poda por dominância: mesmo que o estado parcial seja viável, se seu bound não supera o incumbente, o ramo é abandonado. Essa camada adicional justifica o maior número de podas observado para o Branch and Bound e sua viabilidade em instâncias que excedem os limites práticos do Backtracking.

O custo dessa vantagem temporal é o consumo crescente de memória para manter a fronteira de nós ativos. Para $n = 20$ em Partição, o Branch and Bound utilizou 47 MB, enquanto o Backtracking operou em $O(n)$ de espaço. Essa troca memória por tempo é o fator determinante na escolha entre as duas técnicas para instâncias de porte médio. Para instâncias grandes, ambas tornam-se inviáveis, motivando o uso de técnicas com complexidades mais favoráveis, como Programação Dinâmica e algoritmos gulosos, que serão objeto de análise nas entregas subsequentes.

Em termos de implementação, o Backtracking é mais simples apenas a pilha de recursão e a condição de poda são necessários. O Branch and Bound exige a definição de um bound admissível, uma estrutura de fila de prioridade e a gestão do incumbente, aumentando a complexidade de implementação. Para problemas onde um bound informativo é difícil de definir, como a AGM com custo parcial como único critério, a vantagem prática do Branch and Bound é reduzida.

6 CONCLUSÃO

Esta primeira entrega confirmou a viabilidade de Backtracking e Branch and Bound para os problemas de Partição de Conjuntos e Árvore Geradora Mínima, com correteza preservada em todas as instâncias concluídas. A análise teórica e empírica evidenciou que ambas as técnicas compartilham complexidade exponencial no pior caso, mas diferem significativamente na eficácia prática: o Branch and Bound reduz o espaço explorado por meio de podas por dominância, ao custo de maior consumo de memória pela manutenção da fronteira de nós ativos.

Para o Problema de Partição de Conjuntos, o Branch and Bound demonstrou maior escalabilidade, completando $n = 20$ com 104.474 podas enquanto o Backtracking visitou o mesmo número de nós sem descartá-los antecipadamente. Para a AGM, o Branch and Bound estendeu a viabilidade até $V = 10$, enquanto o Backtracking atingiu o limite prático para o mesmo tamanho. Esses resultados ilustram que a qualidade do bound é o

fator determinante para o desempenho prático do Branch and Bound quando o bound é pouco informativo, como na AGM com custo parcial como único critério, o ganho sobre o Backtracking é reduzido.

As limitações inerentes à complexidade exponencial de ambas as abordagens ficam evidentes no crescimento do tempo e da memória com o tamanho da entrada. A superação dessas limitações motiva a incorporação de Programação Dinâmica e Estratégia Gulosa nas entregas subsequentes, que permitirão comparar paradigmas com perfis de complexidade fundamentalmente distintos pseudo-polinomial e polinomial, respectivamente frente aos mesmos problemas aqui estudados.

7 REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

HOROWITZ, E.; SAHNI, S.; RAJASEKARAN, S. Fundamentals of Computer Algorithms. 2. ed. Hoboken: Silicon Press, 2007.

SKIENA, S. S. The Algorithm Design Manual. 2. ed. New York: Springer, 2008.